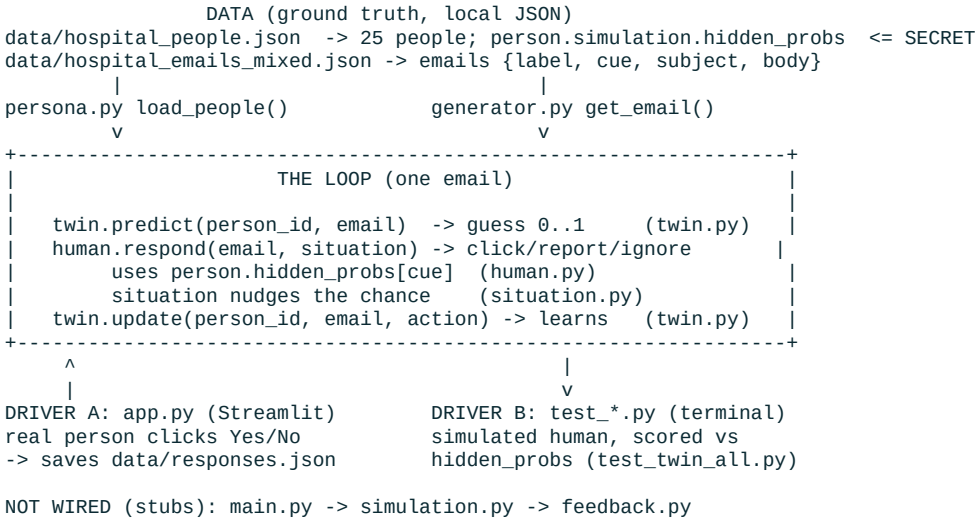
Phishing Digital Twin — Project Architecture & Workflow

How everything connects and works (presentation reference)

In one line

A simulated human acts on emails from **secret** click-probabilities; a **digital twin** learns to predict those actions purely by observation, never seeing the secrets. The project has two working drivers of one shared loop — the Streamlit app (real human) and the test scripts (simulated human) — plus stubs for the planned feedback experiment.

The workflow



1. Entry points

- `app.py` — primary interactive entry; run `python3 -m streamlit run app.py`. Web UI for real Yes/No answers.
- `test_twin.py` — one person, simulated; prints twin guess vs real per cue.
- `test_twin_all.py` — all 25 people; prints per-person accuracy.
- `test_persona.py` / `test_situation.py` / `test_generator.py` — checks of single components.
- `main.py` — intended CLI entry, currently an empty stub (not functional).

2. Core data / control flow (one click in the app)

1. `persona.load_people()` has loaded all 25 people into Persona objects (cached).
2. `generator.get_email()` served a random email from the emails JSON.
3. `twin.predict(person_id, email)` returns the current guess for that email's cue.
4. Yes maps to `click`, No to `ignore`; `twin.update(...)` increments that cue's counter.
5. The answer is appended to `data/responses.json` via save_response().
6. Streamlit reruns; `twin.get_all_estimates(person_id)` feeds the live bar chart.

In the simulated loop (test scripts) the action instead comes from `human.respond(email)`, which reads `person.hidden_probs[cue]` and rolls a die; the run then scores the gap between guess and truth.

3. Module map (one responsibility each)

File	Responsibility
`data/hospital_people.json`	25-person workforce + each person's hidden simulation block (ground truth).
`data/hospital_emails_mixed.json`	The email set: label (phishing/legitimate) + cue + content.
`persona.py`	Persona class + load_people(); only reader of the JSON into objects.
`generator.py`	EmailGenerator: serves emails.
`human.py`	SimulatedHuman.respond(): turns a hidden probability into an action.
`situation.py`	Per-email moment (time, busyness) that nudges behavior.
`twin.py`	TwinPredictor: the model. Counts per person+cue; predict/update. Never reads hidden_probs.
`app.py`	Streamlit UI driver + response logging.
`test_*.py`	Runnable experiments / component checks.
`main.py / simulation.py / feedback.py`	Stubs for the planned full experiment (feedback stage).
`make_people.py`	Stale: generated the OLD 30-person schema; do not run (would overwrite data).

4. Key state & configuration

- **Ground-truth state**: data/hospital_people.json (simulation.hidden_probs) — static, read at load.
- **Learned state**: TwinPredictor.counters (dict keyed by person_id), in-memory, resets each run.
- **Session state**: Streamlit st.session_state holds the twin, email order, position, count across reruns.
- **Persisted output**: data/responses.json — every Yes/No answer logged.
- **Config**: minimal, hard-coded (file paths, NUM_EMAILS, random.seed(42)). No .env or settings file.

5. External dependencies & services

- **Streamlit** — the web UI (only runtime third-party dependency).
- **pandas** — DataFrame for the app's chart.
- **Python standard library** — json, random, os, datetime.
- **No database, network, external API or queue.** All data is local JSON; responses.json is the only datastore.
- Tooling only (not the app): reportlab/pdfplumber (PDFs), pptxgenjs (deck); Git/GitHub for version control.

Say plainly: the engine is twin.py + human.py + the loop; app.py and the test scripts are two drivers of that same loop (real vs simulated human). main.py / simulation.py / feedback.py are placeholders for the next phase, not dead code. GitHub: github.com/vvek17/Digital_twin